Graceful Kubernetes Shutdown

Day 11 | KUBERNETES / EKS / AKS / GKE

Use this guide to investigate a specific failure, apply a cause-specific change and verify recovery. Start with evidence from the affected workload. The examples require adaptation to your environment.

How the failure happens

Shutdown is a distributed sequence. Traffic routing changes propagate while the process is being terminated. The application must stop accepting new work and finish or safely abandon existing work within its configured deadlines.

The Scenario An app exits immediately during replacement while requests are still in flight.

The Gap Pod readiness and rollout completion do not prove all requests drained safely.

The Fix Has Two Parts Application signal handling and sufficient Kubernetes termination time.

Before you start

A test environment with realistic request duration; signal-aware application; suitable entrypoint; access to ingress or load-balancer drain settings; failure metrics.

Diagnostic checks

```
kubectl describe pod <pod>

# Correlate termination times with HTTP errors.
# Inspect SIGTERM handling and LB draining.
```

Check The Process: Use an entrypoint that delivers termination signals to the real app.

Check The Path: Ingress/LB deregistration and endpoint propagation take time.

Apply the cause specific fix

Fix The Confirmed Cause

```
# Pod spec fragment; tune from measurements
terminationGracePeriodSeconds: 45
containers:
- name: api
  image: registry.example.com/api:v1
  lifecycle:
    preStop:
      exec:
        command: ["sh", "-c", "sleep 5"]
```

App Must Cooperate: Handle SIGTERM, stop new work and finish allowed in-flight requests before exiting.

Sleep Is Not A Universal Fix: The hook is a measured propagation buffer. It consumes the same grace period and requires a shell.

Validate the result

```
# Generate traffic in a test environment.
# Trigger a rolling deployment.
# Observe request failures, drain time and exits.
```

Expected: Requests complete within the allowed drain time and the process exits before forced termination.

Long Requests?: Align timeouts, app shutdown and LB drain settings. Decide what to do with unfinished work.

Practice exercise

Send continuous requests during a rollout. Compare termination signals, backend removal, request errors and process exit time. Test one long request deliberately.

Implementation notes

Pod fragment requires a real image with sh and sleep. The hook runs before the normal TERM signal and uses part of terminationGracePeriodSeconds. At expiry remaining processes can be killed. Kubernetes endpoint removal and connection draining are not synchronous across all layers. Distroless images need a suitable alternative. Readiness probes and rollout settings should match the app. Do not use sleep duration as a guarantee.

Interview question

Does a preStop sleep guarantee zero dropped requests? No. It is only a possible propagation buffer and consumes the same termination grace period.

Record what changed

- The symptom and affected user operation.
- Evidence supporting the root cause and alternatives ruled out.
- The changed configuration and the version or identity involved.
- Positive and negative tests, plus any remaining risks.

Example status

Examples are educational templates or fragments. They were not deployed against a live cloud account or production cluster. A successful syntax check is not proof of runtime compatibility. Test adapted configurations in an isolated environment first.

Official references

Official documentation 1